

README: Red-Black Tree Counting Project

项目结构

```
./
├── README.md
├── documents
│   └── report.pdf      # 详细的项目报告
└── code
    ├── BF.cpp          # 解法1: 暴力枚举
    ├── DP.cpp          # 解法2: 动态规划
    ├── GF.cpp          # 解法3: 生成函数 (Karatsuba 乘法)
    ├── NTT.cpp         # 解法4: 生成函数 (NTT 乘法)
    ├── BF.exe          # (编译后生成)
    ├── DP.exe          # (编译后生成)
    ├── GF.exe          # (编译后生成)
    └── NTT.exe         # (编译后生成)
```

如何编译和运行

你需要一个 C++ 编译器（如 `g++`）和一个 C++17 或更高版本的环境（`NTT.cpp` 中的 `__int128` 类型需要）。

1. 进入 `code` 目录

所有源代码都在 `code` 文件夹中。首先，请打开终端并进入该目录：

```
cd code
```

2. 编译

你可以选择编译你想要运行的算法。

选项 1: 编译 `DP.cpp` (动态规划 - $O(N^2 \log N)$)

```
# -o DP.exe 指定输出文件名为 DP.exe
# -O2 开启优化
# -std=c++17 使用 C++17 标准 (log2 需要 <cmath>)
g++ -o DP.exe -g -O2 -std=c++17 DP.cpp
```

选项 2: 编译 GF.cpp (生成函数 - $O(N^{1.58} \log N)$)

```
g++ -o GF.exe -g -O2 -std=c++17 GF.cpp
```

选项 3: 编译 NTT.cpp (最快的解法 - $O(N \log^2 N)$)

```
# NTT.cpp 依赖 __int128 和 <cmath>, 推荐使用 C++17
g++ -o NTT.exe -g -O2 -std=c++17 NTT.cpp
```

选项 4: 编译 BF.cpp (暴力枚举 - $O(8^N)$) 警告: 此算法极其缓慢, 仅适用于 $N < 6$ 的测试。

```
g++ -o BF.exe -g -O2 -std=c++17 BF.cpp
```

3. 运行

编译成功后, `code` 目录中会生成对应的 `.exe` 可执行文件。

运行程序:

```
./DP.exe
```

(或者 `./GF.exe`, `./NTT.exe`, `./BF.exe`)

输入和输出

程序启动后会等待用户输入。

1. 输入一个整数 N (代表内部节点数)。
2. 按 `Enter` 键。
3. 程序将输出一个整数, 即 N 个节点的红黑树数量 (对 1000000007 取模)。

示例:

```
$ ./DP.exe  
5  
8
```

算法简介

- **BF.cpp**: 暴力枚举。生成所有树形态和所有染色方案, 然后验证。
- **DP.cpp**: 动态规划。基于节点数、黑高和根节点颜色构建 DP 状态。
- **GF.cpp**: 生成函数。DP 解法的优化版, 使用 Karatsuba 乘法 ($O(N^{1.58})$) 来加速卷积。
- **NTT.cpp**: 生成函数 (最终版)。使用数论变换 (NTT) 来实现 $O(N \log N)$ 的多项式乘法, 是本项目最快的解法。

详细报告

关于所有算法的详细推导、复杂度分析和性能测试, 请参阅 [documents/report.pdf](#)